SKILL SWAP — FINAL PRESENTATION SCRIPT

Peer-to-Peer Learning Platform • 10–11 Minute Complete Team Script & Defense Sheet

HOST LINK: **http://localhost:3000/**
MODE: 9 Screens Dock

Status: Ready to Present

TOTAL TIME	TEAM MEMBERS	TECH STACK	TEJAS POSITION
10–11 Minutes	Tulika, Varun, Tejas, Ujjwal	React 19 • Node • Mongo • WebRTC	**2nd Last (Slot 3) ■**

1. TULIKA — Opening, College Problem & Barter Vision

0:00 – 2:30 (2.5 Mins)

SCREEN CUE: Show Screen 1 (Landing Page)

"Good morning respected teachers and fellow students. My name is **Tulika**, and along with my teammates **Varun, Tejas, and Ujjwal**, we are proud to present our project: **Skill Swap** — a peer-to-peer collaborative learning platform built for college students.

In college, whenever we want to pick up a new technical or creative skill — like React development, Figma UI/UX, or public speaking — we face two big problems:
1. **Expensive recorded video courses** that cost thousands of rupees, where no instructor answers our personal doubts.
2. **Watching YouTube tutorials alone**, without hands-on practice or individual mentorship.

But if you look around our campus, seniors and batchmates already possess these exact skills! One student is great at coding; another is great at design.
Skill Swap connects these students through a zero-cost barter credit economy.

On our platform, **knowledge is currency**. You do not need real money to learn. When you teach someone a skill you know, you earn **Credits**. You can then spend those credits to learn any skill you want from another student!

We built the frontend using **React.js, Vite, and modern CSS**, adopting a clean Academic Teal theme designed to feel approachable, human-made, and organized.

To explain how our backend securely manages user data, sessions, and credit balances, I now hand over to **Varun**."

Expected Judge Question: Why React instead of HTML/JS?
Your 1-Line Defense: "React's modular components allow us to reuse cards, navbars, and modals seamlessly while updating live state with zero page reloads."

2. VARUN — Backend Architecture, Database Schemas & APIs

2:30 – 5:00 (2.5 Mins)

SCREEN CUE: Show Terminal / Health Check API or Architecture Flow

"Thank you, Tulika. Hello everyone, my name is **Varun**, and I will explain the **Backend Architecture, Database Design, and RESTful APIs** of Skill Swap.

Our backend runs on **Node.js using the Express.js framework**, connected to a **MongoDB database via Mongoose ODM**. We followed a clean MVC architecture to ensure modularity, high performance, and security.

We designed four core database schemas:
1. **User Schema**: Stores student profiles, password hashes, current credit balances, and two distinct arrays: skillsToTeach and skillsToLearn.
2. **Session Schema**: Tracks booking requests, status lifecycles (*requested, accepted, completed*), meeting durations, and WebRTC room IDs.
3. **Feedback Schema**: Captures numerical ratings and evaluation criteria such as clarity, helpfulness, and punctuality.
4. **Gamification Schema**: Stores unlocked badges, learning streaks, and community rankings.

For security, we implemented **JWT (JSON Web Tokens)** for stateless authentication and **bcrypt.js** to salt and hash all passwords.

Through our **Learner and Tutor Module APIs**, our server safely validates credit balances before confirming any booking, locking the amount in escrow until the session is completed so no double-spending can occur.

Now, to walk you through the actual live student experience and booking flow, I hand over to **Tejas**."

Expected Judge Question: How do you prevent credit double-spending?
Your 1-Line Defense: "Our API verifies credit balances before booking, locks the amount in escrow, and only releases it after session status becomes 'completed'."

3. TEJAS — Live Student Journey, Dashboard, Booking & Credits ★ [YOUR PART - 2ND LAST]

5:00 – 7:30 (2.5 Mins)

SCREEN CUE: 1. Click Screen 3 (Dashboard) → 2. Click Screen 6 (Book) → 3. Click Screen 9 (Messages)

"Thank you, Varun! Hello everyone, I'm **Tejas**, and I will walk you through the **live student experience on Skill Swap**.

(Click Screen 3: Student Dashboard)
Once a student signs in, they land right here on their personalized **Student Dashboard**.
At the top, you can see my profile with **250 welcome credits** ready to use. Below that, we have four clear stat cards:
• My upcoming sessions,
• Skills currently in progress,
• Badges I've earned,
• And how many fellow students I've helped so far!

Below that, the system presents **'Recommended for you'** tutor cards — such as Rohan Sharma for Web Development and Priya Singh for UI/UX Design — showing their star ratings and skills.

(Click Screen 6: Book a Session Page)
Now, booking a session with a tutor is completely frictionless. On the **Booking Screen**, a student can choose between:
• A **1-on-1 private session**, which costs **1 credit**.
• Or a **Group study session**, which costs only **0.5 credits** per person.

I simply pick a date on our interactive calendar — for example, October 14th — select an available time slot like 3:00 PM, and click **Confirm Booking**. The session is booked instantly without any back-and-forth emails.

(Click Screen 9: Messages Page)
Before the meeting begins, students can also coordinate directly inside our built-in **Messages screen**, where they can share notes, GitHub repositories, and clarify their doubts in advance.

Now, when it's time for the actual live session to start, how do students connect over real-time video without leaving the website? To explain our in-browser WebRTC video room and AI engine, here is **Ujjwal**."

Expected Judge Question: What happens if a student runs out of credits?
Your 1-Line Defense: "They simply offer a 1-hour session to teach a skill they know, earn credits back from peers, and continue learning!"

4. UJJWAL — WebRTC Video Room, AI Skill Matching & Conclusion

7:30 – 10:00 (2.5 Mins)

SCREEN CUE: 1. Click Screen 7 (Video Call) → 2. Click Screen 8 (Leaderboard)

"Thank you, Tejas. Hello everyone, I'm **Ujjwal**, and I will be explaining our **Real-Time WebRTC Video System, AI Skill Recommendation Engine, and Gamification**.

(Click Screen 7: Video Room Page)
Unlike traditional platforms that force students to switch to Zoom or Google Meet, Skill Swap features **native in-browser video calling powered by WebRTC**.
WebRTC streams encrypted audio and video directly peer-to-peer between browsers with near-zero latency. Our Node.js server acts as the **Signaling Server**, negotiating SDP handshakes and ICE candidates.
As you can see in Screen 7's theater mode, we have a running elapsed session timer, dual camera tiles for tutor and learner, floating call controls (mic, camera, screen share), and a live notes and chat sidebar.

AI Skill Matching Engine:
To intelligently match learners with the right tutors, we designed a Python microservice with **Scikit-learn**. It computes **cosine similarity** between a learner's wanted skills and tutor profiles, ranking mentors by relevance, past ratings, and credibility.

(Click Screen 8: Leaderboard Page)
Finally, to sustain student motivation, our **Gamification Engine** rewards active participation with badges like *First Step, Helper, Consistent, and Explorer*.
Whenever a session ends and feedback is submitted, the tutor's **AI Credibility Score** automatically recalculates based on weighted peer ratings.

In conclusion, Skill Swap is a complete, self-sustaining peer ecosystem that makes skill learning democratic, free, and accessible to every student.

Thank you so much for your time, and our team is now ready for any questions!"

Expected Judge Question: Why use WebRTC over Zoom links?
Your 1-Line Defense: "WebRTC connects the two browsers directly with peer-to-peer encryption, eliminating third-party accounts and meeting link friction."

■■ MASTER TEAM Q&A; DEFENSE CHEAT SHEET

Expected Judge Question	Speaker	Exact 1-Line Response
What happens if a student runs out of credits?	**Tejas**	They offer a 1-hour session to teach what they know, earn credits back from peers, and keep learning.
What if someone cancels a booking?	**Tejas**	The locked credit is refunded to the learner immediately, and the tutor's calendar slot reopens.
Why React instead of HTML/JS?	**Tulika**	React's modular components and hooks allow dynamic state updates without reloading the entire page.
How do you prevent credit double-spending?	**Varun**	Our API verifies balances beforehand, locks credits in escrow, and only releases upon 'completed' status.
How does AI compute credibility?	**Ujjwal**	It dynamically weights completed ratings and review criteria, recalculating after every session feedback.